Phase 1
Engineering Decisions & Development Log

1. Chunk Size and Overlap

Decision

The Phase 1 ingestion pipeline is designed to divide extracted document text into **150-word chunks with a 30-word overlap** between consecutive chunks.

Technical Rationale

The 150-word chunk size was specified in the Phase 1 ingestion design to divide the extracted aquaculture information into manageable text blocks for subsequent processing and retrieval. The selected size provides a defined unit of text while retaining the information contained within the surrounding context.

A **30-word overlap** is included between consecutive chunks to maintain **contextual continuity**. This is particularly relevant for aquaculture information where an operational value, threshold, condition, or qualification may be described across nearby portions of text. The overlap reduces the likelihood that such information becomes separated completely between adjacent chunks.

The Phase 1 plan specifically identifies the purpose of the overlap as ensuring **contextual continuity** and states that the chunking configuration should use 150-word blocks with a 30-word overlap.

Phase 1 Specification

- **Chunk size:** 150 words
- **Overlap:** 30 words
- **Processing stage:** Document ingestion
- **Purpose:** Maintain contextual continuity during text segmentation

2. Embedding Model

Decision

The Phase 1 pipeline uses **BAAI/bge-m3** as the embedding model for converting the processed text chunks into dense vector representations.

Technical Rationale

After the document text has been extracted and divided into chunks, the chunks need to be represented in a form that can be indexed for retrieval. The Phase 1 plan specifies the use of **BGE-M3** for this embedding stage.

The model was selected in the project specification for its suitability for **cross-lingual Indic/English representation** and **multi-granularity retrieval**. This is relevant to AquaAssist because the knowledge corpus is based on the collected English aquaculture documents, while the broader project is intended to support vernacular interaction.

The embedding stage therefore acts as the representation layer between the processed text corpus and the vector database. The Phase 1 workflow specifies that the text blocks are transformed into dense vectors using BGE-M3 before being indexed into ChromaDB.

Phase 1 Specification

- **Embedding model:** `BAAI/bge-m3`
- **Input:** Processed text chunks
- **Output:** Dense vector representations
- **Next stage:** ChromaDB indexing
- **Key design consideration:** Cross-lingual Indic/English representation and multi-granularity retrieval

3. Vector Store

Decision

The Phase 1 pipeline uses **ChromaDB** as the local vector store for the generated embeddings.

Technical Rationale

The embeddings generated from the processed text chunks need to be stored in a persistent vector database so that they can subsequently be used during retrieval.

The Phase 1 implementation specifies a **local ChromaDB instance** for this purpose. The project plan identifies ChromaDB as appropriate for **zero-server, offline persistence within local edge-hardware constraints**.

This choice also keeps the vector database as a locally maintained component of the Phase 1 pipeline rather than introducing a separate database server into the architecture. The embedding script is specified to generate the BGE-M3 vectors and index those vectors into the local ChromaDB instance.

Phase 1 Specification

- **Vector store:** ChromaDB
- **Deployment:** Local
- **Purpose:** Persistent storage of generated vectors

- **Architectural requirement:** Offline / local operation
- **Database location:** `data/chroma_db/`

The repository specification further identifies `data/chroma_db/` as a **strictly local** directory rather than a Git-tracked project artifact.

4. Git Policy

Decision

Raw PDF files and the generated **ChromaDB database** are excluded from Git version control. The processed JSON data generated by the ingestion pipeline is permitted to be tracked.

Technical Rationale

The Phase 1 repository structure separates the original documents, processed data, vector database, source code, and documentation into different directories.

The original PDF corpus is maintained outside the Git repository. The Phase 1 plan explicitly states that raw PDF files **must not be committed** and should remain in the shared Google Drive or on individual local machines.

Similarly, the `chroma_db` directory contains generated database files and indexes and is therefore also excluded from version control. The plan specifies that this database should be generated locally by running the embedding pipeline, or transferred separately when manual sharing is required.

In contrast, the processed text output located in `data/processed/` is intended to be tracked. The plan specifically identifies the processed `.json` files as allowed repository data so that team members can work from the same processed text baseline.

Phase 1 Repository Policy

Component	Location	Git Status
Raw PDF corpus	`data/raw_pdfs/`	**Not tracked**
Processed text	`data/processed/`	**Tracked**
ChromaDB	`data/chroma_db/`	**Not tracked**
Python scripts	`src/`	**Tracked**
Documentation	`docs/`	**Tracked**

This separation is part of the repository architecture defined for Phase 1.

Phase 2
Engineering Decisions & Development Log

1. Phase 2 Objective

Phase 2 extends the Phase 1 knowledge-base and retrieval pipeline by introducing automated RLAIF data generation, NLI-based evidence verification, three-way decision routing, and constrained edge generation. The objective is to improve evidence verification, reduce unsupported responses, and produce structured outputs suitable for offline aquaculture assistance.

2. NLI Cross-Encoder Decision Gate

The Phase 2 pipeline introduces an NLI-based cross-encoder verification stage to evaluate whether the retrieved evidence supports the user's query. The NLI model produces probabilities for three semantic relationships: Entailment, Neutral, and Contradiction. These probabilities are used as the basis for deterministic routing rather than allowing the generation model to decide whether the retrieved evidence is sufficient.

The current decision policy uses the following initial thresholds:

- ANSWER: P(Entailment) ≥ 0.75
- CLARIFY: P(Neutral) > 0.40
- ABSTAIN: P(Contradiction) > 0.30

These thresholds are treated as initial engineering values and will be evaluated empirically through threshold sweeps on the benchmark dataset. The objective is to identify a routing configuration that improves precision and recall while reducing unsupported answers.

Current Status: Initial decision thresholds defined; empirical calibration pending.

3. Three-Way Decision Policy

The NLI verification output is mapped to a deterministic three-way decision policy. Instead of treating verification as a simple pass/fail operation, the system routes each query-evidence pair into one of three states: **ANSWER, CLARIFY, or ABSTAIN**.

- **ANSWER:** Selected when P(Entailment) ≥ 0.75, indicating that the retrieved evidence sufficiently supports the query.
- **CLARIFY:** Selected when P(Neutral) > 0.40, indicating that the available evidence does not provide sufficient support and additional clarification may be required.

- **ABSTAIN:** Selected when P(Contradiction) > 0.30, indicating that the retrieved evidence conflicts with the query and the system should avoid generating an unsupported answer.

These routing conditions provide a deterministic safety layer between evidence retrieval and language-model generation. The thresholds are initial values and will be calibrated using benchmark evaluation and threshold-sweep experiments.

Current Status: Three-way routing policy defined; empirical validation pending.

4. RLAIF Dataset Generation

Phase 2 introduces an automated **RLAIF data-generation pipeline** to create cross-lingual NLI examples from the verified aquaculture knowledge base. The pipeline uses the Phase 1 `chunks.json` as its primary evidence source, ensuring that generated examples remain grounded in the project's curated corpus.

The generated dataset contains three NLI categories:

- **Entailment** — the evidence supports the query.
- **Neutral** — the evidence does not provide sufficient information.
- **Contradiction** — the evidence conflicts with the query.

The pipeline is designed to generate **Telugu query–English evidence pairs**, supporting the cross-lingual NLI verification requirement of AquaAssist. The generated examples are passed through the biological invariant verification stage before being accepted into the dataset.

Input: `data/processed/chunks.json`
Expected output: `data/evaluation/rlaif_dataset.json`

The verified dataset will be used for NLI evaluation, threshold calibration, and development of the three-way **ANSWER / CLARIFY / ABSTAIN** decision policy.

Engineering Rationale: Automating dataset generation reduces manual annotation effort while keeping examples grounded in the project's domain-specific evidence.

Current Status: Pipeline design defined; implementation and validation pending.

5. Biological Invariant Verification

The RLAIF generation pipeline includes a **biological invariant verification layer** to prevent automatically generated training or evaluation examples from containing biologically impossible or unsafe aquaculture values.

The verification function, `verify_aquaculture_invariants`, checks generated examples against predefined domain constraints before they are admitted into the final dataset. The Phase 2 plan specifically identifies checks such as valid **pH ranges of 6.0–9.5** and validation of **dissolved oxygen (DO)** values.

This guardrail is important because RLAIF examples are automatically generated and therefore require an additional domain-level validation step. Invalid examples should be rejected rather than allowing them to influence NLI evaluation or calibration.

Validation flow: Generated example → Biological invariant checks → Accept / Reject → Verified dataset

Engineering Rationale: Prevent domain-invalid synthetic examples from entering the training/evaluation data and reduce the risk of unsafe model behavior.

Current Status: Invariant rules defined; implementation and validation pending.

6. GBNF-Constrained Generation

AquaAssist uses **GBNF (GGML BNF) grammar constraints** to control the structure of responses generated by the local Qwen-2.5-1.5B-Instruct model. Instead of allowing unrestricted text generation, the grammar defines the expected output structure and restricts the model to the required format.

The planned grammar file is:

`constraints.gbnf`

The generated JSON is designed to include the following key fields:

- `evidence_chunk_id` — identifies the supporting knowledge chunk.
- `parameter_identified` — identifies the relevant aquaculture parameter.
- `recommended_action_telugu` — provides the recommended action in Telugu.

This constrained-generation layer is intended to ensure that the model produces **machine-readable, structured output** and maintains an explicit connection between the response and its retrieved evidence. The Phase 2 plan specifies that GBNF will enforce strict JSON structure on the Qwen-2.5-1.5B outputs.

Engineering Rationale: Constraining the output format reduces formatting errors and makes the generated response easier for downstream components and the local UI to validate and display.

Current Status: Output fields and grammar requirement defined; implementation pending.

7. Qwen-2.5-1.5B Edge Generation

AquaAssist is designed to perform language generation locally using the **quantized Qwen-2.5-1.5B-Instruct** model through **llama.cpp Python bindings**. This allows the generation stage to operate on an edge/CPU environment without depending on a cloud-based language model.

The model will receive the verified evidence and the structured generation requirements from the preceding pipeline stages. Its output will be constrained using the `constraints.gbnf` grammar so that the final response follows the required JSON structure.

Key implementation target:

- Model: **Qwen-2.5-1.5B-Instruct**
- Runtime: **llama.cpp Python bindings**
- Model format: **Quantized**
- Generation: **GBNF-constrained**
- Target environment: **Local CPU / edge device**
- Performance target: **TTFT < 3 seconds on CPU**

Engineering Rationale: A small quantized model reduces memory and computational requirements while supporting the project's fully offline architecture.

Current Status: Model and runtime requirements defined; integration and performance testing pending.

8. Architecture Decisions

Phase 2 extends the Phase 1 knowledge-base pipeline by adding **NLI verification, deterministic decision routing, automated RLAIF data generation, and constrained local generation**.

The major architectural decisions are:

- **NLI verification before generation:** Retrieved evidence is evaluated before the language model is allowed to produce the final response.
- **Three-way deterministic routing:** NLI probabilities are mapped to **ANSWER, CLARIFY, or ABSTAIN** using predefined thresholds.
- **Automated RLAIF data generation:** Phase 1 knowledge chunks are reused to create cross-lingual NLI examples.
- **Biological invariant filtering:** Automatically generated examples are checked against aquaculture-specific constraints before dataset admission.
- **Constrained generation:** GBNF is used to enforce the required JSON output structure.
- **Local edge generation:** Quantized Qwen-2.5-1.5B with llama.cpp is selected to support offline CPU-based generation.

These decisions maintain the project's core requirement of **offline, evidence-grounded, and controlled response generation**, while extending the Phase 1 retrieval pipeline toward a complete end-to-end system.

Current Status: Phase 2 architecture defined; individual modules are being implemented and will be integrated as they become available.

9. Experimental Results & Calibration

The initial NLI decision thresholds are treated as engineering starting points and will be validated experimentally using the Phase 2 benchmark dataset. Threshold sweeps will be performed to study how different decision boundaries affect system reliability and coverage.

The main evaluation measures will include:

- **Precision** — correctness of responses accepted by the system.
- **Recall** — ability to identify evidence-supported queries.
- **Selective Risk** — error rate among responses that the system chooses to answer.
- **Safety/Coverage behavior** — trade-off between answering more queries and avoiding unsupported or contradictory responses.

The experiments will compare different threshold configurations for the **ANSWER, CLARIFY, and ABSTAIN** states. The final thresholds will be selected based on empirical performance rather than being assumed to be optimal from the initial design.

Initial Thresholds:
ANSWER → P(Entailment) ≥ 0.75
CLARIFY → P(Neutral) > 0.40
ABSTAIN → P(Contradiction) > 0.30

Current Status: Evaluation framework defined; benchmark execution and threshold calibration pending.

10. Challenges and Solutions

Phase 2 introduces several engineering challenges because multiple components must work together while maintaining the project's offline and safety requirements.

Challenge 1 — Reliable NLI decision boundaries:
Fixed thresholds may not provide the best balance between answering supported queries and avoiding unsupported responses.
Planned solution: Perform systematic threshold sweeps and compare precision, recall, selective risk, and coverage before finalizing the routing thresholds.

Challenge 2 — Quality of automatically generated RLAIF data:
Synthetic examples may contain incorrect or biologically impossible aquaculture information.
Planned solution: Apply `verify_aquaculture_invariants` before accepting generated examples into the dataset.

Challenge 3 — Structured local generation:
A small language model may produce outputs that do not follow the required response format.
Planned solution: Use the `constraints.gbnf` grammar to enforce the required JSON structure and fields.

Challenge 4 — CPU inference performance:
Local generation must remain practical on CPU-based edge hardware.
Planned solution: Use the quantized Qwen-2.5-1.5B-Instruct model with llama.cpp and measure TTFT, with a target of **less than 3 seconds**.

Current Status: Challenges and corresponding mitigation strategies have been identified. Actual implementation issues and their empirical solutions will be added during development.

11. Weekly Development Log

Week	Date	Work / Decision	Technical Details	Status / Outcome
Week 1	08 Sep 2026	Phase 2 setup and literature organization	Created `EAV-RAG Phase 2` Zotero collection and organized Phase 2 literature covering NLI, RAG, RLAIF, DPO, embeddings, hallucination detection, and quantization.	Completed
Week 1	08 Sep 2026	Engineering logbook initialization	Created Phase 2 logbook sections covering NLI verification, three-way decision policy, RLAIF generation, biological invariants, GBNF generation, edge inference, architecture, evaluation, challenges, and weekly tracking.	Completed
Week 1	08 Sep 2026	Initial NLI routing policy documented	Recorded initial thresholds: Entailment ≥ 0.75, Neutral > 0.40, Contradiction > 0.30.	Defined; calibration pending